



University of Mindanao

College of Computing Education



Course: Automata and Language Theory

Members:

Abas, Emman

Asan, Nino

Bravo, Joana

Hingpit, Hazeljoy

Nanding, Nazlah

Instructor:

Modesto C. Tarrazona

Submission Date:

Table of Contents

1. Introduction
2. Objectives
3. Background and Literature Review
4. Methodology
5. Implementation
 - o Deterministic Finite Automata (DFA)
 - o Non-Deterministic Finite Automata (NDFA)
 - o Pushdown Automata (PDA)
 - o Context-Free Grammars (CFG)
 - o Tower of Hanoi
 - o Turing Machines
6. Results and Analysis
7. Conclusion
8. Future Work
9. References
10. Appendix (if any)

Introduction

Automata and language theory is a branch of theoretical computer science that deals with the study of abstract machines and the formal languages they recognize. This field forms the foundation for understanding computation and is vital in areas like programming languages, compilers, and artificial intelligence (Sipser, 1996). This project is a unified platform for compiling and executing various laboratory activities from the CST4 Automata and Language Theory course. These activities cover key computational topics such as Deterministic Finite Automata (DFA), Non-Deterministic Finite Automata (NFA), Pushdown Automata (PDA), Context-Free Grammar (CFG), the Tower of Hanoi, and Turing Machines. The project's importance lies in its ability to combine these theoretical concepts and algorithms into a single, accessible system with a graphical user interface (GUI). This integration provides a hands-on tool for students to better understand and explore the fundamental computational concepts behind automata and problem-solving techniques.

Objectives

The project aims to create a system that simplifies and organizes laboratory activities, giving both students and educators a practical way to study and work with automata and computational models. Combining these activities into one platform makes it easier to revisit and test the concepts, helping students better understand the theory while also applying it in real-world scenarios. This hands-on approach strengthens knowledge and encourages experimentation and problem-solving more interactively and efficiently.

Background and Literature Review

Our project focuses on automata theory and its applications, implemented in Java with a graphical user interface (GUI) to enhance user interaction. The program provides five main modules—Deterministic and Nondeterministic Finite Automata (DFA/NFA), Pushdown Automata (PDA), Context-Free Grammar (CFG), Turing Machine, and the Tower of Hanoi—each accessible through designated buttons. Upon selecting a module, the corresponding program launches, prompting the user for input to demonstrate theoretical concepts in a practical context. Automata theory is central to computational science, dealing

with abstract machines and formal languages. DFA and NFA are foundational models used to recognize regular languages, differing in their transition mechanisms, with DFAs allowing unique transitions and NFAs enabling multiple transitions for a single input. PDAs expand this capability by introducing a stack, which allows recognition of context-free languages, commonly associated with recursive structures such as balanced parentheses. CFGs further define these languages through production rules, which are essential in syntax analysis and language parsing.

Turing Machines represent the most powerful theoretical model of computation, capable of simulating any algorithm and addressing problems of decidability and complexity. The inclusion of the Tower of Hanoi module, while not directly part of automata theory, illustrates problem-solving through recursion and algorithmic efficiency. This project consolidates previous works, including simulations of DFA/NFA transitions for pattern recognition, implementations of PDA operations for validating context-free structures, CFG parsers for syntax analysis, Turing Machine algorithms for computational tasks, and recursive solutions to the Tower of Hanoi problem. By integrating these components into a single GUI-based program, the project not only highlights the practical applications of automata theory but also provides an intuitive platform for exploring theoretical concepts in computational science.

Methodology

Our project focuses on automata theory and its applications, which are implemented as a graphical user interface (GUI) program developed using Java. The program integrates five key topics—Deterministic and Nondeterministic Finite Automata (DFA/NFA), Pushdown Automata (PDA), Context-Free Grammar (CFG), Turing Machine, and the Tower of Hanoi—into a single, cohesive platform. Users interact with the program through a GUI featuring button-based navigation, where selecting a topic launches its corresponding module and prompts the user for input. The project consolidates previously developed programs on these topics, transforming them into a unified application for learning and exploration.

The methodology centers on applying core principles of automata and computation theory. DFA/NFA modules implement transition tables and state diagrams to simulate string

recognition in regular languages. PDA modules use stack-based algorithms for handling context-free languages, adhering to standard grammar parsing techniques. CFG modules utilize production rules and recursive algorithms to validate or generate language strings. Turing Machine modules simulate computational processes through state transitions, following the principles of algorithmic computation and decidability. The Tower of Hanoi module demonstrates recursive problem-solving with an efficient divide-and-conquer approach.

The project leverages Java as the primary programming language due to its versatility and robust GUI capabilities, employing tools such as Swing or JavaFX for designing the user interface. Flowcharts were used during the design phase to map program logic, illustrating user interactions, transitions between modules, and input-output flows for each component. These diagrams provided clarity in organizing the integration of individual modules, ensuring consistency and functionality across the application. Through this structured approach, the project delivers an interactive and user-friendly tool for understanding automata theory and computational models.

Implementation

A. Deterministic Finite Automata (DFA)

Define the Problem or Task Modeled Using DFA

This program will evaluate various user-defined inputs, such as numbers, letters, or words, and determine whether each input is valid or invalid based on specific rules.

The DFA is designed to validate and classify input strings into specific categories:

Numbers: Strings consisting only of digits (e.g., "123").

Single Letters: Strings containing a single alphabetic character (e.g., "a").

Words: Strings made up entirely of alphabetic characters (e.g., "hello").

This is a common task in software applications for **input validation**, where user-provided data is checked for correctness.

Formal Description

- A DFA is formally described as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- **States (Q)** : {q0, q1, q2, q3}.
 - q0: Initial state.
 - q1: Accepts valid numbers.
 - q2: Accepts valid single letters.
 - q3: Accepts valid words.
- **Alphabet (Σ)**: {a-z, A-Z, 0-9}.
- **Transition Function (δ)**: Maps states and inputs to next states:
 - $\delta(q0, \text{digit}) = q1$.
 - $\delta(q0, \text{letter}) = q2$.
 - $\delta(q2, \text{letter}) = q3$.
- **Start State (q0)**: The state where all inputs begin.
- **Accept States (F)**: {q1, q2, q3}.

Transition Logic:

The DFA processes the input string character by character:

- At state q0:
 - If a digit is encountered, transition to state q1.
 - If a letter is encountered, transition to state q2.
- At state q2:
 - If subsequent letters are encountered, remain in state q3.

Any input that does not follow these rules (e.g., mixing digits and letters like "1a") leads to rejection.

1. GUI Representation:

- Our DFA_NFA_GUI program accepts user input via a text field.
- Outputs are displayed in a results area, showing whether the input is a valid number, letter, or word.

2. Examples:

Example 1:

Input: "123"

- Start at q0.
- Read '1' → Transition to q1.
- Read '2' → Stay in q1.
- Read '3' → Stay in q1.

- End at q1 → Output: "Valid Number".

Example 2:

Input: "hello"

- Start at q0.
- Read 'h' → Transition to q2.
- Read 'e', 'l', 'l', 'o' → Transition to q3.
- End at q3 → Output: "Valid Word".

Example 3:

Input: "1a"

- Start at q0.
- Read '1' → Transition to q1.
- Read 'a' → Invalid transition → Output: "Invalid".

The image displays three side-by-side screenshots of a web application titled "DFA and NFA SIMULATOR". Each screenshot shows the input field, the results section, and the "Validate" and "Clear" buttons.

- Left Screenshot:** Input is "123". Results show "DFA Result: Valid Number" and "NFA Result: Valid Number".
- Middle Screenshot:** Input is "hello". Results show "DFA Result: Valid Word" and "NFA Result: Valid Word".
- Right Screenshot:** Input is "1a". Results show "DFA Result: Invalid" and "NFA Result: Invalid".

Figures 1, 2 and 3. Sample Output of DFA Simulator when input is accepted and rejected

The figure above demonstrates the output of a **DFA** (Deterministic Finite Automaton) simulator, validating whether a given string meets predefined criteria. The program determines if the input qualifies as a "Valid Word," "Valid Number," or "Invalid" based on DFA rules.

The three examples showcase the program's results for different inputs:

1. Input: "hello" → The DFA identifies this as a "Valid Word" because it matches the condition for a valid sequence of letters.

2. Input: **"123"** → The DFA classifies this as a "Valid Number" since it contains only numeric characters.
3. Input: **"1sdds"** → The DFA marks this input as "Invalid" because it does not meet the criteria for a valid word or number.

The program interface allows users to input strings and validate them against DFA rules. Results are displayed in the output area, with clear labels indicating whether the DFA accepts or rejects the input.

B. Non-Deterministic Finite Automata (NFA)

Define the Problem or Task Modeled Using NFA

This program evaluates user-defined inputs such as numbers, letters, or words and determines whether each input is valid or invalid based on predefined rules. The NFA introduces non-deterministic behavior, allowing multiple possible transitions for a given input from the same state. This flexibility enables the validation of numbers, single letters, and words efficiently.

The NFA accepts the following categories of inputs:

- Numbers: Strings consisting only of digits (e.g., "123").
- Single Letters: Strings containing a single alphabetic character (e.g., "a").
- Words: Strings made up entirely of alphabetic characters (e.g., "hello").

Formal Description

An NFA is formally described as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- **States (Q):** $\{q_0, q_1, q_2, q_3\}$.
 - q_0 : Start state.
 - q_1 : Accepts valid numbers.
 - q_2 : Intermediate state for single letters.
 - q_3 : Accepts valid words.
- **Alphabet (Σ):** $\{a-z, A-Z, 0-9\}$.
- **Transition Function (δ):** Maps states and inputs to possible next states:
 - $\delta(q_0, \text{digit}) \rightarrow \{q_1\}$.
 - $\delta(q_0, \text{letter}) \rightarrow \{q_2, q_3\}$ (*nondeterministic transition*).
 - $\delta(q_2, \text{letter}) \rightarrow \{q_3\}$.
- **Start State (q_0):** All inputs begin at this state.
- **Accept States (F):** $\{q_1, q_2, q_3\}$.

How NFA Differs from DFA

- **Nondeterministic Behavior:**

An NFA allows multiple possible transitions for the same input. For example, when a letter is entered at q_0 , the NFA transitions to both q_2 (intermediate state) and q_3 (accept state for words).

- **Parallel Path Exploration:**

The NFA simultaneously explores all valid paths to determine if the input can be accepted. In contrast, a DFA strictly follows one unique path for each input.

Describe the Implementation Process

Transition Logic

The program implements the NFA behavior using regular expressions to simulate nondeterministic transitions:

- A string of digits (e.g., "123") transitions from $q_0 \rightarrow q_1$ and is classified as a Valid Number.
- A single alphabetic character (e.g., "a") transitions from $q_0 \rightarrow q_2$ and is classified as a Valid Single Letter.
- Multiple alphabetic characters (e.g., "hello") transition from $q_0 \rightarrow q_3$ and are classified as a Valid Word.
- Any combination of letters and digits that do not follow these rules is marked as Invalid.

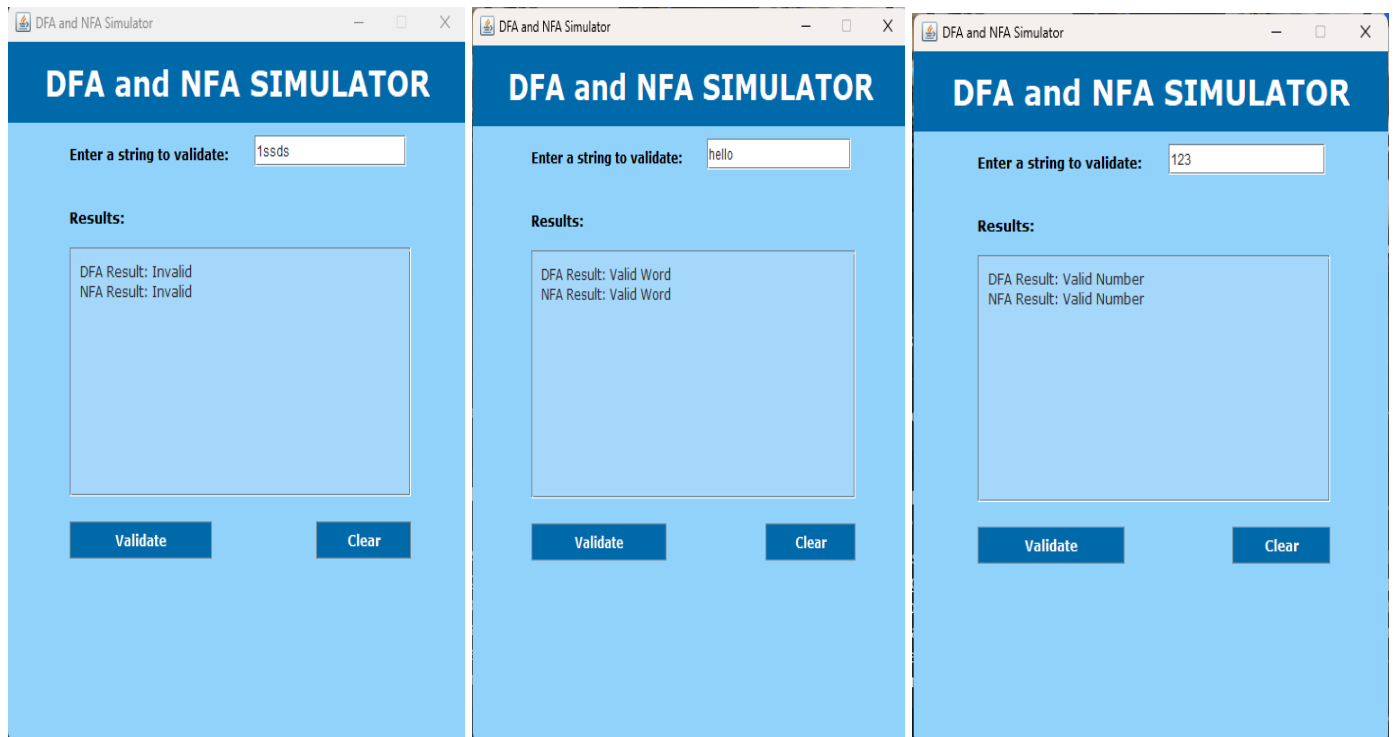
GUI Representation

The graphical user interface (GUI) accepts user input via a text field and outputs results in a dedicated results area. This interface allows users to validate inputs against NFA rules and receive clear feedback.

Examples

The following examples demonstrate the behavior of the NFA:

1. Input: "hello"
 - Transition Path: $q_0 \rightarrow q_3$
 - Output: "Valid Word"
2. Input: "123"
 - Transition Path: $q_0 \rightarrow q_1$
 - Output: "Valid Number"
3. Input: "1sdds"
 - Transition Path: $q_0 \rightarrow \text{Invalid}$
 - Output: "Invalid"



Figures 4, 5, and 6. Sample Output of NFA Simulator when input is accepted and rejected

The figures illustrate the outputs of the NFA simulator, validating whether the input qualifies as a **"Valid Word," "Valid Number,"** or **"Invalid."**

- Input: **"hello"**
 - The NFA transitions to q_3 and identifies the input as a Valid Word.
- Input: **"123"**
 - The NFA transitions to q_1 and identifies the input as a Valid Number.
- Input: **"1sdds"**
 - The NFA explores transitions but cannot reach an accept state, marking the input as Invalid.

C. Pushdown Automata (PDA)

Define the Problem or Task Modeled Using PDA

This PDA-based program validates strings that follow the context-free language $L = \{a^n b^n \mid n \geq 1\}$, where each string consists of exactly n occurrences of the symbol 'a' followed by n occurrences of the symbol 'b'. The program also tests for other context-free languages such as palindromic strings with a center delimiter, $L = \{wcw^r \mid w \in \Sigma^*, c \in \Sigma\}$, where the reverse of w appears after the center symbol 'c'.

Formal Description

A Pushdown Automaton (PDA) is formally described as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- **States (Q):** $\{q_0, q_1, q_f\}$
- **Input Alphabet (Σ):** $\{a, b, c\}$
- **Stack Alphabet (Γ):** $\{a, b, Z_0\}$
- **Transition Function (δ):**
 - Push 'a': $\delta(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$
 - Pop 'a' for 'b': $\delta(q_1, b, a) \rightarrow (q_1, \epsilon)$
 - Transition to final state: $\delta(q_1, \epsilon, Z_0) \rightarrow (q_f, \epsilon)$
- **Start State (q_0):** Initial state
- **Initial Stack Symbol (Z_0):** Bottom of the stack
- **Accept State (F):** $\{q_f\}$

How the PDA Works

1. **Push Phase:** Each occurrence of the symbol 'a' is pushed onto the stack.
2. **Pop Phase:** For every 'b' in the input, the PDA pops one 'a' from the stack.
3. **Acceptance:** The input is accepted if the stack is empty when the string ends.
4. **Rejection:** The input is rejected if the symbols are unequal, the stack underflows, or invalid characters appear.

Describe the Implementation Process

1. Stack Mechanism

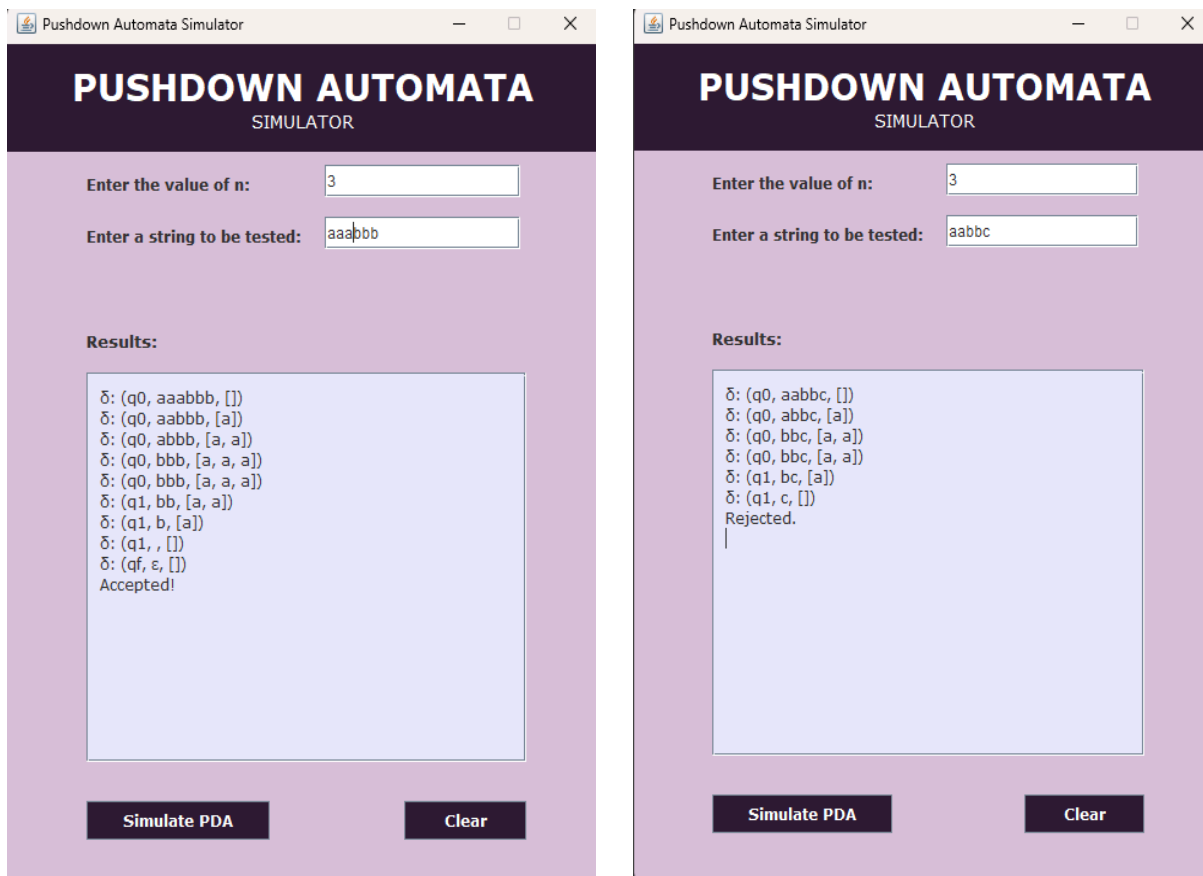
- The PDA is implemented using Java's Stack class to simulate the stack's push and pop operations.
- The input string is processed one character at a time:
 - Push each 'a' onto the stack.
 - Pop one 'a' for each 'b'.
- If the stack becomes empty at the end of the input string, the program transitions to the accept state.

2. GUI Representation

- A simple PDA_GUI interface is designed with the following components:
 - Input fields for "Enter the value of n" and "Enter a string".
 - An output text area that displays the stack operations and final validation results.
- The program visually represents the stack transitions step-by-step for user clarity.

3. Examples

Input	Process	Output
"aaabbb"	Push 3 'a's → Pop 3 'b's → Stack empty	"Accepted"
"aabbc"	Unequal 'a's and 'b's → Extra 'c' detected	"Rejected"
"abcba"	Palindromic check → Stack symmetry validated	"Accepted"



Figures 7 and 8. Sample Output of PDA Simulator when input is accepted and rejected

The figures demonstrate the program's ability to validate strings for specific conditions:

1. **Accepted Example:** Input: *"aaabbb"*
 - Push 3 'a's → Pop 3 'b's → Stack becomes empty → Output: "Accepted".
2. **Rejected Example:** Input: *"aabbc"*
 - Unequal number of 'a's and 'b's → Extra 'c' invalidates input → Output: "Rejected".
3. **Palindromic Example:** Input: *"abcba"*
 - Stack symmetry confirms the input matches a palindrome pattern → Output: "Accepted".

D. Context-Free Grammar (CFG)

Define the Problem or Task Modeled Using CFG

Generate and verify strings based on the context-free grammar (CFG) $L = \{wcwR \mid w \in \{a,b\}^*\}$. The task is to determine which strings are accepted or rejected by the grammar, gaining an understanding of how CFGs handle strings.

Formal Description

Context-free grammars (CFGs) are formal grammars used in theoretical computer science and linguistics to characterize language syntax. Compilers, parsers, and programming languages are all designed using CFGs. By outlining guidelines for how strings can be created from a starting symbol, they determine the syntax of strings that are part of a language.

Context-free grammar G can be defined by four tuples as:

$$G = (V, T, P, S)$$

Where:

G describes the grammar

T describes a finite set of terminal symbols.

V describes a finite set of non-terminal symbols

P describes a set of production rules

S is the start symbol.

In CFG, the start symbol is used to derive the string. You can derive the string by repeatedly replacing a non-terminal by the right-hand side of the production until all non-terminals have been replaced by terminal symbols.

In the activity, the language $L = \{wcwR \mid w \in \{a,b\}^*\}$, which is made up of strings where w is any combination of a and b , followed by a c , and then w^R , the opposite of w , is represented by the grammar. The alphabet is $\Sigma = \{a, b\}$ means the language consists of strings formed by the letters a and b . The notation $w \in (a, b)^*$ indicates that w is a string composed of any combination (including the empty string) of the letters a and b . On the other hand, w represents any string that can be formed using the alphabet $\{a, b\}$. This string can be of any length, including zero length (i.e., the empty string). In addition, wR denotes the reverse of the string w . For example, if $w = "ab"$, then $wR = "ba"$. The expression $\{wcwR \mid w \in (a, b)^*\}$ describes a set of strings where each string is formed by concatenating w , followed by the character c , followed by the reverse of w (wR).

This CFG can be beneficial in showcasing how context-free grammars handle palindromic patterns and concatenations. Compilers, automata theory, and computational linguistics all use it to identify or produce structured patterns in a language.

Description of CFG given the language $L = \{WCWR \mid W \in \{a,b\}^*\}$:

- $T = \{a,b,c\}$: These are the terminal symbols that form the strings in the language.
- $N = \{S, W\}$: N represents the non-terminal symbols. S is the start symbol, and W generates the sequence w , representing any string that can be formed using the alphabet $\{a, b\}$. This string can be of any length, including zero length (i.e., the empty string).

Production rules:

$$S \rightarrow aSa$$
$$S \rightarrow bSb$$
$$S \rightarrow c$$

Implementation Process

The figure displays two side-by-side screenshots of a web-based simulator titled "CONTEXT-FREE GRAMMAR SIMULATOR".

Left Screenshot (Accepted):

- Input:** "Enter a string to be tested:" followed by a text box containing "abacaba".
- Results:** A box containing the text "The string 'abacaba' is accepted by the CFG."
- Buttons:** "Simulate CFG" and "Clear".

Right Screenshot (Rejected):

- Input:** "Enter a string to be tested:" followed by a text box containing "aabcaa".
- Results:** A box containing the text "The string 'aabcaa' is rejected by the CFG."
- Buttons:** "Simulate CFG" and "Clear".

Figure 9. Sample Output of Context-Free Grammar when input is accepted and rejected

The figure above shows the sample output of context-free grammar when input is accepted or rejected. The given Java program checks to see if an input string complies with

certain requirements to imitate a Context-Free Grammar (CFG). It establishes two conditions: WE, in which a string is composed solely of the characters "a" and "b," and WCWR, in which a string has a center "c" and the left half is the opposite of the right half. In a loop, the software takes user input, uses the isAccepted method to determine whether the string satisfies either criterion and reports whether it is accepted or rejected. The program is organized with distinct validation methods (isWCWR and isWE), guaranteeing clear and modular logic, and it runs until the user writes "exit."

E. Tower of Hanoi

Define the Problem or Task Modeled Using the Tower Of Hanoi

To create a Java program to solve the Tower of Hanoi problem. The program will ask the user to enter the number of disks (n) and then show the steps to move all the disks from the starting rod/pole to the target rod/pole using a helper rod/pole.

Formal Description

Three towers, or pegs, plus many rings, make up this mathematical puzzle called the Tower of Hanoi. These rings are stacked in ascending order, meaning that the smaller one rests on top of the larger one, and they vary in size. In other puzzle variations, the tower count stays the same but the number of disks increases.

A well-known example of a recursive algorithm that illustrates automata and computational theory concepts is the Tower of Hanoi problem. It has the following connections to these ideas: Recursive Process as computing Model, Algorithmic problem Solving, State Transition, and Efficiency and Complexity.

Implementation Process

Key Components of the Code

Attributes:

- **numDisks:** Stores the number of disks to be solved in the problem.
- **rods:** A List of List<Integer> representing the three rods/poles and the disks on each rod.
- **canvas:** A JPanel that handles the graphical representation of the rods and disks.
- **diskInputField:** A text field where the user inputs the number of disks.

- **startButton and resetButton:** Buttons for starting and resetting the simulation.

Graphical Interface:

- The GUI is constructed using JFrame with three main sections:
 - **Title Panel:** Displays the title of the application.
 - **Control Panel:** Contains controls for user input (diskInputField) and action buttons (startButton, resetButton).
 - **Canvas Panel:** The main area for drawing rods and disks.

Functionality

a. Initialization

- The initializeRods method sets up the rods. It clears all rods and adds all disks (from largest to smallest) to the first rod (rods.get(0)).

b. Drawing Rods and Disks

- drawRods: Draw the three rods on the canvas.
- drawDisks: Dynamically calculates the size and position of each disk for visual appeal such as the disk size is proportional to its value (smaller disks have narrower widths) and each disk has a unique shade of yellow for better differentiation.

c. Event Handling

- Start Button: Reads the number of disks from diskInputField and validates input and reinitializes the rods and starts the simulation.
- Reset Button: Resets the number of disks to 3 (default) and redraws the rods/disks.

Recursive Solution Approach

The rule of the problem is that only one disk can be moved at a time. A disk can only be placed on top of a larger disk or an empty rod. Lastly, all disks must be moved from the source rod to the target rod using a helper rod. The problem must be broken down into smaller subproblems to be solved: Using the target rod, move $n-1$ disks from the source rod to the helper rod. Proceed straight from the source rod to the target rod with the largest disk (n). Using the source rod, move the $n-1$ disks from the helper rod to the target rod. Up until

the base case ($n=1$), in which a single disk is transferred straight from the source rod to the destination rod, this recursive procedure is carried out.

In the source code, the moveDisk method visually updates the rods by removing the top disk from the source rod and adding it to the destination rod, repainting the canvas after each move, and adding a delay (Thread.sleep) for smoother animation.

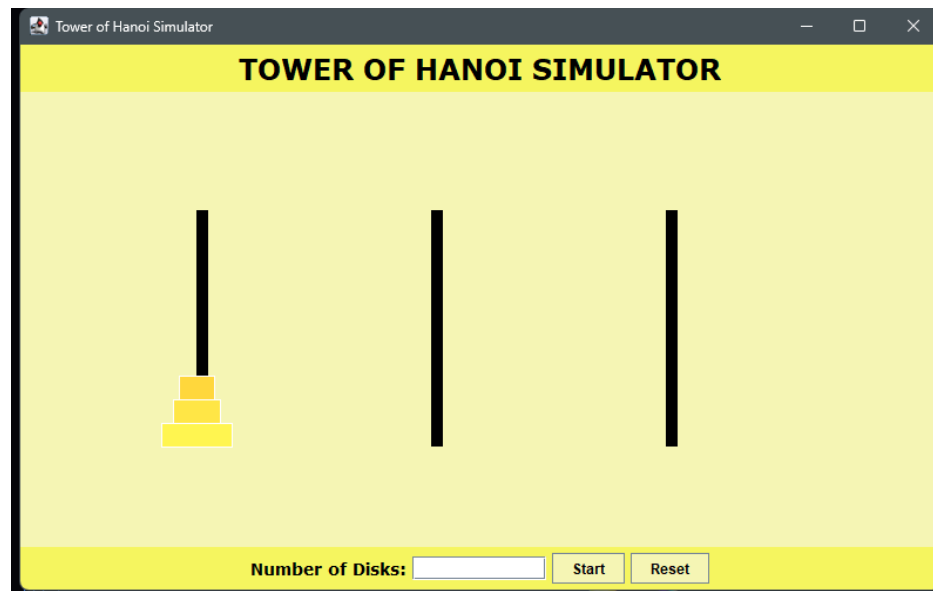


Figure 10. Graphical User Interface of 'Tower Of Hanoi Simulator'

Figure 10 shows the graphical user interface of 'Tower of Hanoi Simulator.' There are three vertical black rods, each representing a pole (common in the Tower of Hanoi problem). On the leftmost rod, there are yellow disks of decreasing size, stacked from largest at the bottom to smallest at the top.

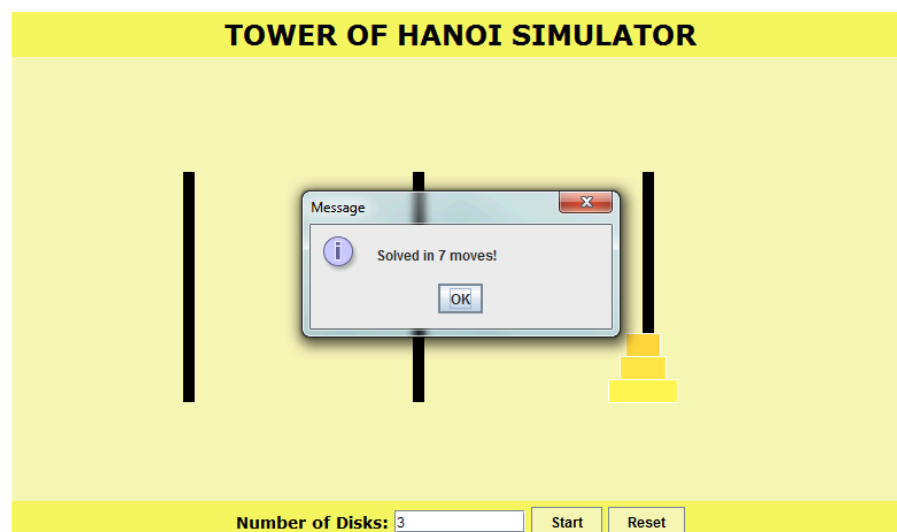


Figure 11. The output of Tower of Hanoi Simulator When User Input 3

The Tower of Hanoi Simulator's sample output is shown in Figure 11 when the user enters 3 for the number of disks. The simulation starts when the user enters a number of disks and hits the start button. Following the simulation, a message indicating how many steps or moves are needed to solve the Tower of Hanoi will be displayed.

```
Enter the number of disks: 3

Steps to solve the Tower of Hanoi:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Solved in 7 moves.
```

Figure 12. Console form of Tower of Hanoi

Figure 12, reveals the console form of the Tower of Hanoi and shows the movement of the disks from the three vertical rods. It also illustrates how the problem is recursive and how it relates to ideas in computational theory such as state transitions, recursion, and algorithmic problem resolution.

F. Turing Machines

Define the Problem or Task Modeled Using the Turing Machine

The activity assigned to us involves designing and implementing a Turing Machine model capable of performing two distinct operations such as addition and multiplication or division, and subtraction and multiplication or division.

Formal Description

A mathematical model known as a Turing machine (TM) is made up of an infinitely long tape that is divided into cells and that receives input. It has a head that reads the tape input. The Turing machine's state is kept in a state register. An input symbol is read, altered internally, and then moved from one cell to the right or left before being replaced by another symbol. The input string is accepted if the TM reaches the end state; if not, it is rejected.

A Turing machine can be formally described as a 7-tuple $(Q, X, \Sigma, \delta, q_0, B, F)$ where,

Q is a finite set of states,

X/T is the tape alphabet,

Σ is the input alphabet,

δ/Δ is a transition function; $\delta : Q \times X \rightarrow Q \times X \times \{\text{Left_shift}, \text{Right_shift}\}$,

q_0 is the initial state,

B is the blank symbol,

F is the set of final states.

Theoretically, any calculation that can be carried out algorithmically can be simulated using a Turing machine. It is necessary to specify the input, the intended output, and the procedure or rules utilized to reach that outcome when modeling a problem or task using a Turing machine.

Implementation Process

The goal of our project is to create and develop a Turing Machine model that can carry out binary addition and unary multiplication.

- **Binary Addition:** The Turing Machine will take as input two binary numbers separated by a delimiter (e.g., #) and compute their sum. The result will be written in binary form and as well as in decimal form after the machine halts.
- **Unary Multiplication:** The Turing Machine will multiply two unary numbers, provided as input in unary representation, (1 for each number), and output their product in unary format. Decimal form is also included in the result.

This activity aims to illustrate the computational capability of Turing machines and to formally model these operations, making sure that the outputs and machine transitions follow the predetermined criteria for each operation.

Binary Addition

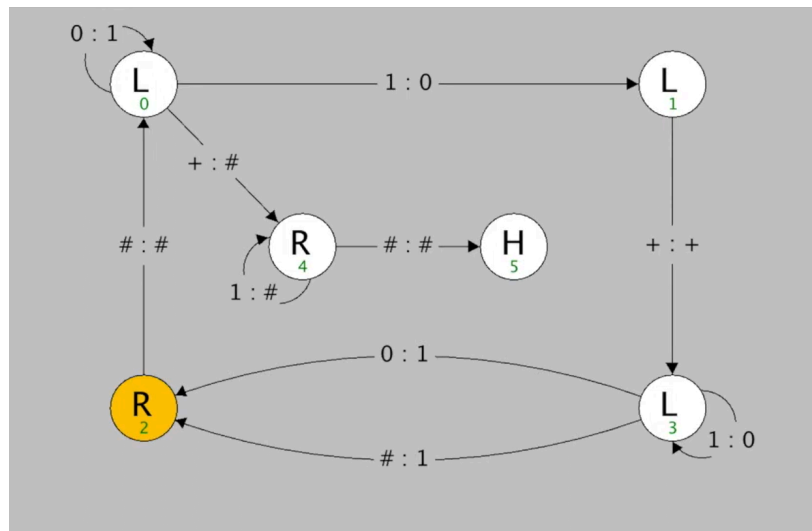


Figure 13. State Diagram of Binary Addition

Figure 13 shows a state diagram of binary addition in a Turing machine. A finite automata or Turing machine with six states—L₀, L₁, L₃, R₂, R₄, and a stopping state H₅—is depicted in this state diagram. Input-output pairs (such as 0:1 or 1:0), in which the machine reads an input and executes an operation, establish state transitions. States R ("Right") and L (probably "Left") entail loops and motions in response to particular circumstances, like coming across 0, 1, or #. Starting at L₀, the machine processes inputs, changes states, and carries out repeated tasks like verifying conditions or overwriting values. When certain conditions are satisfied, it stops at H₅. Its current activity or significance in the process is shown by the highlighted condition R₂. Until it reaches the halting state, the machine reads, writes, and manipulates binary input.

BINARY ADDITION
TURING MACHINE SIMULATION

Enter First Binary Number: 11

Enter Second Binary Number: 11

Results:

- [1]1+11
- 1[1]+11
- [1]10

Final Binary Result: 110
Final Decimal Result: 6

Calculate Clear

Figure 14. Sample Output of Binary Addition

Figure 14 shows the sample output in a graphical user interface of binary addition in a Turing machine highlighting the process and mathematical concept being shown. Two fields are provided where users can input the first and second binary values in the text fields. When the user presses calculate, it will trigger the binary addition process. The binary addition simulation uses the idea of a Turing machine to show how two binary numbers are added. For instance, the Turing machine analyzes the binary inputs step-by-step when the user enters "11" and "11" (both in binary format). It does addition using binary arithmetic rules as it moves over each bit of the binary strings. It manages to carry bits while progressively shifting and storing the outcomes. The final binary result and final decimal result are among the outcomes, where the binary result is translated into its decimal counterpart. In this case, "110" is equivalent to 6. This simulation uses straightforward tape transitions to demonstrate how a Turing machine solves a binary addition problem.

Unary Multiplication

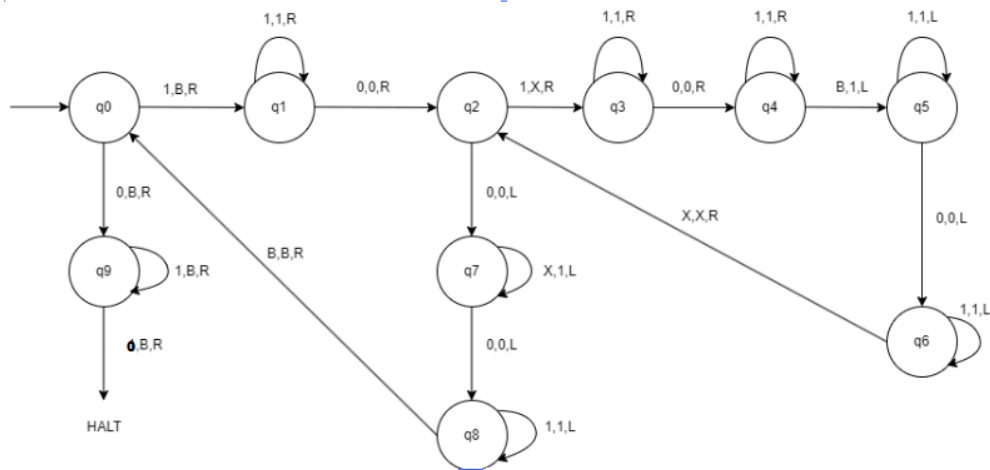
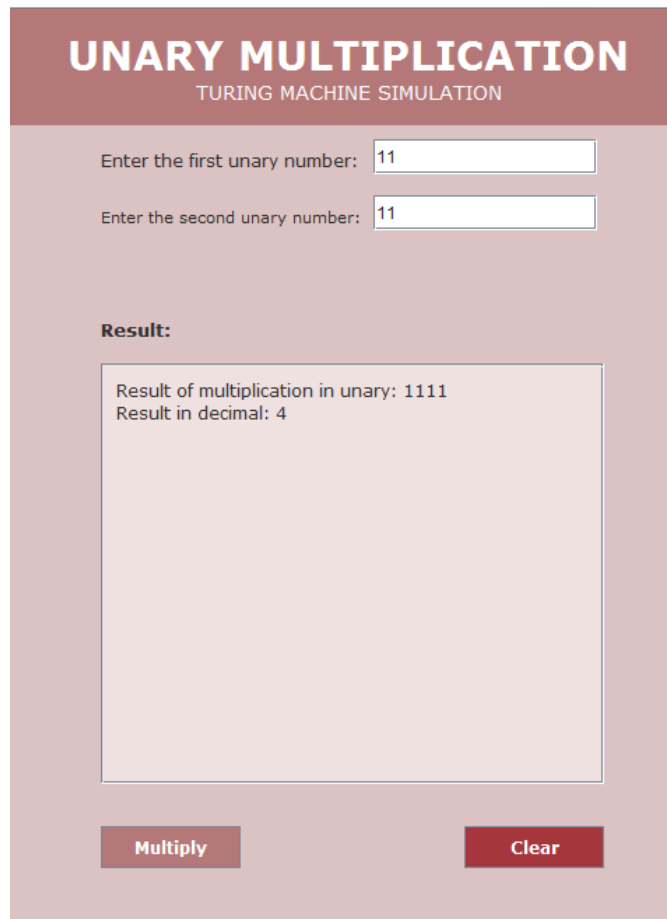


Figure 15. State Diagram of Unary Multiplication

Figure 15 presents the state diagram of the unary multiplication of a Turing machine. Unary representation is a simple format where a number n is represented as n repeated symbols 1. State q_0 is the initial state, if it encounters a 1, then the machine will change it to B which denotes the blank symbol, then it will move to the right transitioning to q_1 . In q_1 , if it encounters another 1, then it will not change the number and it will go to the right. As observed, it loops into q_1 due to the number of 1's that need to be checked until it reaches the separator '0'. It will transition to q_2 if it encounters a separator and will continue to go to the right. 1 will be replaced by a special symbol denoted as 'X' and will continue to the right, transitioning to q_3 . In the diagram, there's a loop where if it encounters a 1, then it will continue to go to the right until it reaches the separator. If it encounters a blank symbol, it will change the blank symbol into 1 then the head will move to the left. After encountering 0, it will not change and it will move to the left. As shown in the diagram, there's a loop in q_6 , where if it encounters 1, it will change the number and will continue to move to the left until it reaches the special symbol 'X'. It will change the X into 1 and it will traverse until all the results are complete and written on the tape.



The image shows a web-based graphical user interface for a Turing Machine simulation titled "UNARY MULTIPLICATION". The interface has a dark red header with the title in white. Below the header, there are two input fields for unary numbers, both containing "11". A "Result:" section displays the output: "Result of multiplication in unary: 1111" and "Result in decimal: 4". At the bottom, there are two buttons: "Multiply" and "Clear".

UNARY MULTIPLICATION
TURING MACHINE SIMULATION

Enter the first unary number:

Enter the second unary number:

Result:

Result of multiplication in unary: 1111
Result in decimal: 4

Figure 16. Sample Output of Unary Multiplication

Figure 16 shows the sample output of unary multiplication in the graphical user interface. The unary multiplication simulation shows how to use a Turing machine to multiply two unary numbers. For instance, the numerals "11" and "11" stand for 2 and 2, respectively. The 1's in unary stand for the quantity or number. Here, "11" corresponds to the decimal value 2. The Turing machine repeatedly concatenates unary strings to perform multiplication. The second number is replicated by each unary unit in the first number. "1111" (4 in unary) is the result of multiplying "11" (2) by "11" (2). When the product is represented as a unary string, the results contain re. "1111" is the outcome in this case. Additionally, the decimal representation of the unary result—four—is transformed. This simulation demonstrates how Turing machines can perform basic arithmetic operations like multiplication using unary notation, showcasing a fundamental concept of computational models.

Results and Analysis

The project successfully created a system that simplifies and organizes laboratory activities for automata and computational models, making it easier for both students and

teachers to use. The project helped students get more engaged with automata theory. By designing, building, and testing automata models, students were able to actively participate in the learning process, which made the subject more interesting and encouraged them to dive deeper into the material.

The interactive features of the platform also made it easier for students to understand automata theory. They could experiment with different models and see the results in real time, helping them connect theoretical concepts with real-world applications. Additionally, the platform gave students the chance to apply what they had learned to practical problems, which helped improve their problem-solving and critical-thinking skills.

This system also simplifies the workflow for both students and instructors by organizing resources, experiments, and analyses in one place. This made the process more efficient and allowed everyone to focus more on learning. It also made it easier for students to access the materials and tutorials they needed, improving the overall learning experience.

While the project met its goals, there were some challenges. Some of us had a learning curve with the system's interface, showing that more detailed documentation and tutorials would be helpful. The system currently focuses on basic automata models, so expanding it to include more complex models and algorithms would make it more useful for students.

Looking ahead, there are several improvements to consider. The user interface can be made more intuitive to help students navigate it more easily. The platform can also be expanded to include more advanced models and algorithms for higher-level learners. Finally, integrating the platform with existing learning management systems would make it easier for students and instructors to access and manage resources, improving the overall experience.

Conclusion

In conclusion, the project successfully created a system that simplifies laboratory activities for automata and computational models, improving student engagement and understanding. It allowed students to connect theory to practice, enhance problem-solving skills, and streamline workflows for both students and instructors. While the platform met its objectives, challenges such as a learning curve and limited scope were encountered. Future

improvements include refining the interface, expanding the models, and integrating the platform with existing learning systems to enhance usability and educational value.

Future Work

Future work on this project could focus on improving the user interface and adding more features. For example, users could input more complex examples for DFA, N DFA, PDA, CFG, the Tower of Hanoi, and Turing Machines, with detailed step-by-step solutions. The project could also include interactive features like visualizations of automata and grammar transitions to help users understand these concepts better. Future research could explore new computational models or improve algorithms for faster processing. Adding more advanced topics in automata theory and computational models could make the project a more useful educational tool for students and teachers.

References

Sipser, M. (1996). Introduction to the Theory of Computation. *ACM Sigact News*, 27(1), 27-29.

Appendix

//Main Screen

```
package project;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;

public class FinalProjectGUI extends
JFrame implements ActionListener {
    private static JLabel backgroundimg;
    private static JPanel loginPanel;
    private static JButton startButton,
    exitButton;
    private static Image bg;
    private static String imgbg = "UM
    image.png";
    public static void main(String[] args) {
        new FinalProjectGUI().setVisible(true);}
    FinalProjectGUI() {
        this.setSize(700, 450);
        this.setTitle("Welcome Module");
        this.setDefaultCloseOperation(EXIT_ON_
        CLOSE);
        this.setLocationRelativeTo(null);
        loginPanel = new JPanel();
        loginPanel.setBackground(Color.black);
        this.getContentPane().add(loginPanel);
        loginPanel.setLayout(null);
        bg = new
        ImageIcon(FinalProjectGUI.class.getRes
        ource(imgbg)).getImage().getScaledInst
        ance(700, 415,
        Image.SCALE_SMOOTH);

        // Start Button
        startButton = new JButton("Start") {

        @Override
```

```
        protected void
        paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D)
            g.create();
            g2.setRenderingHint(RenderingHints.KE
            Y_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
            setOpaque(false);
            setContentAreaFilled(false);
            g2.setColor(getBackground());
            g2.fillRoundRect(0, 0, getWidth(),
            getHeight(), 30, 30);
            super.paintComponent(g2);
            g2.dispose(); } };
        startButton.setFont(new Font("Arial",
        Font.BOLD, 15));
        startButton.setBounds(538, 315, 100,
        30);
        startButton.setBackground(Color.white);
        startButton.setBorder(new
        EmptyBorder(5, 5, 5, 5));
        startButton.addActionListener(new
        ActionListener() {
            @Override
            public void actionPerformed(ActionEvent
            e) {
                dispose();
                new Menu().setVisible(true); } });
        loginPanel.add(startButton);
        exitButton = new JButton("Exit") {
            @Override
            protected void
            paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D)
                g.create();
                g2.setRenderingHint(RenderingHints.KE
                Y_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON);
```

```
                setOpaque(false);
                setContentAreaFilled(false);
                g2.setColor(getBackground());
                g2.fillRoundRect(0, 0, getWidth(),
                getHeight(), 30, 30);
                super.paintComponent(g2);
                g2.dispose(); } };
        exitButton.setFont(new Font("Arial",
        Font.BOLD, 15));
        exitButton.setBounds(538, 350, 100,
        30);
        exitButton.setBackground(Color.white);
        exitButton.setBorder(new
        EmptyBorder(5, 5, 5, 5));
        exitButton.addActionListener(new
        ActionListener() {
            @Override
            public void
            actionPerformed(ActionEvent e) {
                System.exit(0);
            }
        });
        loginPanel.add(exitButton);
        backgroundimg = new JLabel("");
        backgroundimg.setIcon(new
        ImageIcon(bg));
        backgroundimg.setBounds(0, 0, 700,
        417);
        loginPanel.add(backgroundimg); }
        @Override
        public void
        actionPerformed(ActionEvent e) { }}
```

//Menu Screen

```
package project;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
public class Menu extends JFrame {

    private static JPanel menuPanel;
    private static JLabel backgroundImg;
    private static JButton exitButton;
    private static String imgBg = "bg
    image.png";
    public static void main(String[] args) {
        new Menu().setVisible(true);
```

```

}
Menu() {
    this.setSize(950, 600); // Adjusted
    height to give space for header
    this.setTitle("Welcome Module");
    this.setDefaultCloseOperation(EXIT_ON_
CLOSE);
    this.setLocationRelativeTo(null);
    menuPanel = new JPanel();
    menuPanel.setBackground(Color.BLACK)
;
    this.getContentPane().add(menuPanel);
    menuPanel.setLayout(null);
    Image bg = new
    ImageIcon(Menu.class.getResource(img
Bg)).getImage().getScaledInstance(950
, 550, Image.SCALE_SMOOTH);
    backgroundImg = new JLabel(new
    ImageIcon(bg));
    backgroundImg.setBounds(0, 0, 950,
550);
    JLabel header = new JLabel("CST4L
ACTIVITIES");
    header.setFont(new Font("Tahoma",
Font.BOLD, 30));
    header.setForeground(Color.WHITE);
    header.setBounds(330, 50, 300,
40); menuPanel.add(header);
    JLabel folder1 =
    createFolderButton("folder1.png", 150,
100, () ->
    openNewWindow("DFA/NFA"));
    JLabel folder2 =
    createFolderButton("folder2.png", 380,
100, () ->
    openNewWindow("Context-free
Grammar"));
    JLabel folder3 =
    createFolderButton("folder3.png", 600,
100, () -> openNewWindow("Pushdown
Automata"));

    JLabel folder4 =
    createFolderButton("folder4.png", 260,
275, () -> openNewWindow("Turing
Machine"));
    JLabel folder5 =
    createFolderButton("folder5.png", 490,
275, () -> openNewWindow("Tower of
Hanoi"));

    JLabel label1 =
    createFolderLabel("DFA/NFA", 185,
230);
    JLabel label2 =
    createFolderLabel("Context-free
Grammar", 373, 230);
    JLabel label3 =
    createFolderLabel("Pushdown
Automata", 600, 230);
    JLabel label4 =
    createFolderLabel("Turing Machine",
275, 400);
    JLabel label5 =
    createFolderLabel("Tower of Hanoi",
505, 400);
    exitButton = new JButton("Exit") {
        @Override
        protected void
        paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D)
            g.create();
            g2.setRenderingHint(RenderingHints.KE
            Y_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
            setOpaque(false);
            setContentAreaFilled(false);
            g2.setColor(getBackground());
            g2.fillRoundRect(0, 0, getWidth(),
            getHeight(), 35, 30);
            super.paintComponent(g2);
            g2.dispose(); } };
    exitButton.setFont(new Font("Arial",
Font.BOLD, 17));
    exitButton.setBounds(709, 482, 160,
39); // Adjusted position
    exitButton.setBackground(Color.WHITE)
;
    exitButton.setForeground(Color.BLACK);
    exitButton.setBorder(new
    EmptyBorder(5, 5, 5, 5));
    exitButton.addActionListener(new
    ActionListener() {

        @Override
        public void actionPerformed(ActionEvent
        e) {
            System.exit(0);
        }
    });
    menuPanel.add(folder1);
    menuPanel.add(folder2);
    menuPanel.add(folder3);
    menuPanel.add(folder4);
    menuPanel.add(folder5);
    menuPanel.add(label1);
    menuPanel.add(label2);
    menuPanel.add(label3);
    menuPanel.add(label4);
    menuPanel.add(label5);
    menuPanel.add(exitButton);
    menuPanel.add(backgroundImg);

    private JLabel createFolderButton(String
    imagePath, int x, int y, Runnable
    onClickAction) {
        ImageIcon defaultIcon = new
        ImageIcon(
        new
        ImageIcon(Menu.class.getResource(ima
        gePath)).getImage().getScaledInstance(
        115, 115, Image.SCALE_SMOOTH));
        ImageIcon hoverIcon = new
        ImageIcon(
        new
        ImageIcon(Menu.class.getResource(ima
        gePath)).getImage().getScaledInstance(
        130, 130, Image.SCALE_SMOOTH));
        JLabel label = new JLabel(defaultIcon);
        label.setBounds(x, y, 140, 140);
        label.addMouseListener(new
        MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent
            e) {
                label.setIcon(hoverIcon);
                label.setBounds(x - 7, y - 7, 155, 155);
            }
        });
        @Override
        public void mouseExited(MouseEvent e)
        {
            label.setIcon(defaultIcon);
            label.setBounds(x, y, 140, 140); }
        @Override
        public void mouseClicked(MouseEvent
        e) {
            onClickAction.run();
        }
    };
    return label;
}
private JLabel createFolderLabel(String
    text, int x, int y) {
    JLabel label = new JLabel(text);
    label.setFont(new Font("Tahoma",
    Font.BOLD, 14));
    label.setForeground(Color.WHITE);
    label.setBounds(x, y, 250, 30); return
    label;
}
private void openNewWindow(String
    folder) {
    if ("Pushdown
    Automata".equals(folder)) {
        project.PDAGUI.main();
    } else if ("DFA/NFA".equals(folder)) {
        project.NFA_DFA_GUI.main();
    } else if ("Turing
    Machine".equals(folder)) {
        String[] options = {"Addition",
        "Multiplication"};
        int choice =
        JOptionPane.showOptionDialog(null,
        "Choose Operation",
        "Turing Machine",
        JOptionPane.DEFAULT_OPTION,
        JOptionPane.INFORMATION_MESSAGE,
        null,
        options,
        options[0]);
        if (choice == 0) {
            project.TuringMachineGUI.main();
        } else if (choice == 1) {
            project.UnaryMultiplication.main();
        }
    }
    } else if ("Context-free
    Grammar".equals(folder)) {
        project.CFGGUI.main();
    }
    } else if ("Tower of
    Hanoi".equals(folder)) {
        project.TowerOfHanoiVisual.main();
    }
    } else {
        JFrame newWindow =
        new JFrame(folder);
        newWindow.setSize(400, 300);

        newWindow.setDefaultCloseOperation(JF
        rame.DISPOSE_ON_CLOSE);

        newWindow.setLocationRelativeTo(this);

        newWindow.setAlwaysOnTop(true);

        newWindow.add(new
        JLabel("Welcome to " + folder,
        SwingConstants.CENTER),
        BorderLayout.CENTER);
        newWindow.setVisible(true);
    }
}
//NFA and DFA
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class DFANFAGUI extends JFrame
{
    private JTextField inputField;
    private JTextArea outputArea;
    private JButton validateButton;
    private JButton clearAllButton;
    public DFANFAGUI() {
        setTitle("DFA and NFA Simulator");
        setSize(500, 600);

        setDefaultCloseOperation(JFrame.DISPO
        SE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        JPanel headerPanel = new
        JPanel();
        headerPanel.setBackground(new
        Color(0x016aae));
        headerPanel.setLayout(new
        BoxLayout(headerPanel,
        BoxLayout.Y_AXIS));

        headerPanel.setBorder(BorderFactory.cr
        eateEmptyBorder(15, 0, 15, 0));
        add(headerPanel,
        BorderLayout.NORTH);
        JLabel title = new JLabel("DFA and
        NFA SIMULATOR");
        title.setForeground(Color.white);
        title.setFont(new Font("Verdana",
        Font.BOLD, 28));

        title.setAlignmentX(Component.CENTER
        _ALIGNMENT);
        headerPanel.add(title);
        add(headerPanel,
        BorderLayout.NORTH);

        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(null);
        mainPanel.setBackground(new
        Color(0x93d2fd));

        JLabel inputLabel = new
        JLabel("Enter a string to validate:");
        inputLabel.setFont(new
        Font("Verdana", Font.BOLD, 12));

        inputLabel.setForeground(Color.BLACK);
        inputLabel.setBounds(65, 10, 200,
        30);
        mainPanel.add(inputLabel);

        inputField = new JTextField();
        inputField.setBounds(260, 10, 160,
        25);
        mainPanel.add(inputField);

        JLabel resultsLabel = new
        JLabel("Results:");

```

```

        resultsLabel.setFont(new
Font("Verdana", Font.BOLD, 13));

resultsLabel.setForeground(Color.BLACK
);
    resultsLabel.setBounds(65, 60,
200, 30);
    mainPanel.add(resultsLabel);

    JPanel roundedPanel = new
JPanel() {
        @Override
        protected void
paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d =
(Graphics2D) g.create();

            g2d.setRenderingHint(RenderingHints.K
EY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            g2d.setColor(new
Color(0xD8BFD8));
            g2d.fillRoundRect(0, 0,
getWidth(), getHeight(), 30, 30);

            g2d.setColor(Color.DARK_GRAY);
            g2d.drawRoundRect(0, 0,
getWidth() - 1, getHeight() - 1, 30, 30);
            g2d.dispose();
        }
    };

    roundedPanel.setLayout(new
BorderLayout());
    roundedPanel.setBounds(65, 100,
360, 200);
    mainPanel.add(roundedPanel);

    outputArea = new JTextArea();
    outputArea.setFont(new
Font("Verdana", Font.PLAIN, 12));

    outputArea.setBorder(BorderFactory.cre
ateEmptyBorder(10, 10, 10, 10));
    outputArea.setBackground(new
Color(0xA7D9FB));
    outputArea.setEditable(false);
    roundedPanel.add(new
JScrollPane(outputArea),
BorderLayout.CENTER);
    validateButton = new
JButton("Validate");
    validateButton.setFont(new
Font("Verdana", Font.BOLD, 12));
    validateButton.setBounds(65, 320,
150, 30);
    validateButton.setBackground(new
Color(0x016AAE));

    validateButton.setForeground(Color.whit
e);
    validateButton.setFocusable(false);
    mainPanel.add(validateButton);

    clearAllButton = new
JButton("Clear");
    clearAllButton.setFont(new
Font("Verdana", Font.BOLD, 12));
    clearAllButton.setBounds(325,
320, 100, 30);
    clearAllButton.setBackground(new
Color(0x016AAE));

    clearAllButton.setForeground(Color.white
);
    clearAllButton.setFocusable(false);
    mainPanel.add(clearAllButton);

    add(mainPanel,
BorderLayout.CENTER);

    // Button Action
    validateButton.addActionListener(e
-> {
        String input =
inputTextField.getText().trim();
        if (input.isEmpty()) {
            JOptionPane.showMessageDialog(this,
"Please enter a string!", "Input Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        outputArea.setText("");
        outputArea.append("DFA Result: " +
evaluateDFA(input) + "\n");
        outputArea.append("NFA
Result: " + evaluateNFA(input) + "\n");
    });

    clearAllButton.addActionListener(new
ActionListener() {
        @Override
        public void
actionPerformed(ActionEvent e) {
            // Clear all input fields and the
process area
            inputField.setText("");
            outputArea.setText("");
        }
    });

    private String evaluateDFA(String
input) {
        if (input.matches("\\d+") {
            return "Valid Number";
        } else if
(input.matches("[a-zA-Z]")) {
            return "Valid Letter";
        } else if
(input.matches("[a-zA-Z]+")) {
            return "Valid Word";
        } else {
            return "Invalid";
        }
    }

    private String evaluateNFA(String
input) {
        if (input.matches("^ [a-zA-Z]$")) {
            return "Valid Letter";
        } else if
(input.matches("^ [a-zA-Z]+$")) {
            return "Valid Word";
        } else if
(input.matches("^ [0-9]+$")) {
            return "Valid Number";
        } else {
            return "Invalid";
        }
    }

    public static void main() {
        SwingUtilities.invokeLater(() -> {
            DFANFAGUI frame = new
DFANFAGUI();
            frame.setVisible(true);
        });
    }
}
//PDA

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.Stack;

public class PDAGUI extends JFrame {
    private JTextField inputNField,
inputStringField;
    private JTextArea outputArea;
    private JButton simulateButton;
    public PDAGUI() {

        setTitle("Pushdown Automata
Simulator");        setSize(500, 690);

        setDefaultCloseOperation(JFrame.DISPO
SE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        JPanel headerPanel = new
JPanel();
        headerPanel.setBackground(new
Color(0x301934));
        headerPanel.setLayout(new
BoxLayout(headerPanel,
BoxLayout.Y_AXIS));

        headerPanel.setBorder(BorderFactory.cr
eateEmptyBorder(15, 0, 15, 0));
        add(headerPanel,
BorderLayout.NORTH);

        JLabel title = new
JLabel("PUSHDOWN AUTOMATA");
        title.setForeground(Color.white);
        title.setFont(new Font("Verdana",
Font.BOLD, 28));

        title.setAlignmentX(Component.CENTER
_ALIGNMENT);

        JLabel subtitle = new
JLabel("SIMULATOR");

        subtitle.setForeground(Color.white);
        subtitle.setFont(new
Font("Verdana", Font.PLAIN, 14));

        subtitle.setAlignmentX(Component.CEN
TER_ALIGNMENT);

        headerPanel.add(title);
        headerPanel.add(subtitle);
        add(headerPanel,
BorderLayout.NORTH);

        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(null);
        mainPanel.setBackground(new
Color(0xD8BFD8));

        JLabel inputNLabel = new
JLabel("Enter the value of n:");
        inputNLabel.setFont(new
Font("Verdana", Font.BOLD, 12));
        inputNLabel.setBounds(260, 10,
200, 30);
        mainPanel.add(inputNLabel);

        inputNField = new JTextField();
        inputNField.setBounds(260, 10,
160, 25);
        mainPanel.add(inputNField);

        JLabel inputStringLabel = new
JLabel("Enter a string to be tested:");
        inputStringLabel.setFont(new
Font("Verdana", Font.BOLD, 12));
        inputStringLabel.setBounds(65,
50, 200, 30);
        mainPanel.add(inputStringLabel);

        inputStringField = new
JTextField();
        inputStringField.setBounds(260,
50, 160, 25);
        mainPanel.add(inputStringField);

        JLabel results = new
JLabel("Results:");
        results.setFont(new
Font("Verdana", Font.BOLD, 13));
        results.setBounds(65, 130, 200,
30);
        mainPanel.add(results);

        JPanel roundedPanel = new
JPanel() {
            @Override
            protected void
paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2d =
(Graphics2D) g.create();

                g2d.setRenderingHint(RenderingHints.K
EY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                g2d.setColor(new
Color(0xD8BFD8));

```

```

        g2d.fillRoundRect(0, 0,
getWidth(), getHeight(), 30, 30);

g2d.setColor(Color.DARK_GRAY);
g2d.drawRoundRect(0, 0,
getWidth() - 1, getHeight() - 1, 30, 30);
g2d.dispose();
    }
};

roundedPanel.setLayout(new
BorderLayout());
roundedPanel.setBounds(65, 170,
360, 300);
mainPanel.add(roundedPanel);

outputArea = new
JTextArea();
outputArea.setFont(new Font("Verdana",
Font.PLAIN, 12));

outputArea.setBorder(BorderFactory.cre
ateEmptyBorder(10, 10, 10, 10));
outputArea.setBackground(new
Color(0xE6E6FA));
outputArea.setEditable(false);
roundedPanel.add(new
JScrollPane(outputArea),
BorderLayout.CENTER);

JButton simulateButton = new
JButton("Simulate PDA");
simulateButton.setFont(new
Font("Verdana", Font.BOLD, 12));
simulateButton.setBounds(65,
500, 150, 30);

simulateButton.setBackground(new
Color(0x301934));

simulateButton.setForeground(Color.whit
e);

simulateButton.setFocusable(false);
mainPanel.add(simulateButton);

JButton clearAllButton = new
JButton("Clear");
clearAllButton.setFont(new
Font("Verdana", Font.BOLD, 12));
clearAllButton.setBounds(325,
500, 100, 30);
clearAllButton.setBackground(new
Color(0x301934));

clearAllButton.setForeground(Color.white
);
clearAllButton.setFocusable(false);
mainPanel.add(clearAllButton);

add(mainPanel,
BorderLayout.CENTER);

simulateButton.addActionListener(e
-> {
    String nText =
inputNField.getText().trim();
    String testString =
inputStringField.getText().trim();

    if (nText.isEmpty() ||
testString.isEmpty()) {

JOptionPane.showMessageDialog(this,
"Please fill in all fields!", "Input Error",
JOptionPane.ERROR_MESSAGE);
return;
    }

    try {
        int n =
Integer.parseInt(nText);
        outputArea.setText("");
        simulatePDA(n, testString);
    } catch
(NumberFormatException ex) {

JOptionPane.showMessageDialog(this,
"Invalid input for n. Please enter a valid
integer.", "Input Error",
JOptionPane.ERROR_MESSAGE);
    }
});

clearAllButton.addActionListener(new
ActionListener() {

```

```

@Override
public void
actionPerformed(ActionEvent e) {

    inputNField.setText("");
    inputStringField.setText("");
    outputArea.setText("");
}

}

private void simulatePDA(int n, String
testString) {
    Stack<Character> stack = new
Stack<>();
    String state = "q0";
    int i = 0;

    outputArea.append("δ: (" + state
+ ", " + testString + ", [])\n");
    while (i < testString.length()) {
        char currentChar =
testString.charAt(i);

        if (state.equals("q0")) {
            if (currentChar == 'a') {
                stack.push('a');
                outputArea.append("δ:
(q0, " + testString.substring(i + 1) + ",
" + stack + ")\n");
            } else if (currentChar == 'b')
{
                state = "q1";
                outputArea.append("δ:
(q0, " + testString.substring(i) + ", " +
stack + ")\n");
                break;
            } else {

outputArea.append("Rejected: Invalid
symbol.\n");
                return;
            }
        }
        i++;
    }
    while (i < testString.length()) {
        char currentChar =
testString.charAt(i);

        if (state.equals("q1")) {
            if (currentChar == 'b') {
                if (!stack.isEmpty()) {
                    stack.pop();
                    outputArea.append("δ:
(q1, " + testString.substring(i + 1) + ",
" + stack + ")\n");
                } else {

outputArea.append("Rejected: Stack not
empty or input invalid.\n");
                    return;
                }
            } else {

outputArea.append("Rejected.\n");
                    return;
                }
        }
        i++;
    }

    if (state.equals("q1") &&
stack.isEmpty() && i ==
testString.length()) {
        outputArea.append("δ: (qf, ε,
[])\n");
    }

    outputArea.append("Accepted!\n");

    } else {
        outputArea.append("Rejected:
Stack not empty or input invalid.\n");
    }
}

public static void main() {
    SwingUtilities.invokeLater(() -> {

```

```

PDAGUI frame = new
PDAGUI();

frame.setVisible(true);    });
}}
//CFG
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class CFGGUI extends JFrame {
    private JTextField inputStringField;
    private JTextArea outputArea;

    public CFGGUI() {

        setTitle("Context-Free Grammar
Simulator");
        setSize(500, 690);

        setDefaultCloseOperation(JFrame.DISPO
SE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        JPanel headerPanel = new
JPanel();
        headerPanel.setBackground(new
Color(0x013220));
        headerPanel.setLayout(new
BoxLayout(headerPanel,
BoxLayout.Y_AXIS));

        headerPanel.setBorder(BorderFactory.cr
eateEmptyBorder(15, 0, 15, 0));
        add(headerPanel,
BorderLayout.NORTH);

        JLabel title = new
JLabel("CONTEXT-FREE GRAMMAR");
        title.setForeground(Color.white);
        title.setFont(new Font("Verdana",
Font.BOLD, 28));

        title.setAlignmentX(Component.CENTER
_ALIGNMENT);

        JLabel subtitle = new
JLabel("SIMULATOR");

        subtitle.setForeground(Color.white);
        subtitle.setFont(new
Font("Verdana", Font.PLAIN, 14));

        subtitle.setAlignmentX(Component.CEN
TER_ALIGNMENT);

        headerPanel.add(title);
        headerPanel.add(subtitle);
        add(headerPanel,
BorderLayout.NORTH);

        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(null);
        mainPanel.setBackground(new
Color(0x98FB98));

        JLabel inputStringLabel = new
JLabel("Enter a string to be tested:");
        inputStringLabel.setFont(new
Font("Verdana", Font.BOLD, 12));
        inputStringLabel.setBounds(65,
50, 200, 30);
        mainPanel.add(inputStringLabel);

        inputStringField = new
JTextField();
        inputStringField.setBounds(260,
50, 160, 25);

        inputStringField.setBorder(BorderFacto
r.createLineBorder(Color.DARK_GRAY, 1,
true));
        mainPanel.add(inputStringField);

        JLabel results = new
JLabel("Results:");
        results.setFont(new
Font("Verdana", Font.BOLD, 13));
        results.setBounds(65, 130, 200,
30);
        mainPanel.add(results);

```

```

        JPanel roundedPanel = new
        JPanel() {
            @Override
            protected void
            paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2d =
                (Graphics2D) g.create();

                g2d.setRenderingHint(RenderingHints.K
                EY_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON);
                g2d.setColor(new
                Color(0xE0FFE0));
                g2d.fillRoundRect(0, 0,
                getWidth(), getHeight(), 30, 30);

                g2d.setColor(Color.DARK_GRAY);
                g2d.drawRoundRect(0, 0,
                getWidth() - 1, getHeight() - 1, 30, 30);
                g2d.dispose();
            }
        };

        roundedPanel.setLayout(new
        BorderLayout());
        roundedPanel.setBounds(65, 170,
        360, 300);
        mainPanel.add(roundedPanel);

        outputArea = new JTextArea();
        outputArea.setFont(new
        Font("Verdana", Font.PLAIN, 12));

        outputArea.setBorder(BorderFactory.cre
        ateEmptyBorder(10, 10, 10, 10));

        outputArea.setBackground(new
        Color(0xE0FFE0));

        outputArea.setEditable(false);

        roundedPanel.add(new
        JScrollPane(outputArea),
        BorderLayout.CENTER);

        JButton simulateButton = new
        JButton("Simulate CFG");
        simulateButton.setFont(new
        Font("Verdana", Font.BOLD, 12));
        simulateButton.setBounds(65,
        500, 150, 30);

        simulateButton.setBackground(new
        Color(0x013220));

        simulateButton.setForeground(Color.whit
        e);

        simulateButton.setFocusable(false);

        simulateButton.setBorder(BorderFactory.
        createLineBorder(Color.DARK_GRAY, 1,
        true));

        simulateButton.addActionListener(new
        ActionListener() {
            @Override
            public void
            actionPerformed(ActionEvent e) {
                String inputString =
                inputStringField.getText();
                if (inputString.isEmpty()) {

                    outputArea.setText("Please enter a
                    string to simulate.");
                    return;
                }

                if
                (CFG.isAccepted(inputString)) {
                    outputArea.setText("The
                    string \"\" + inputString + "\"\" is accepted
                    by the CFG.");
                } else {
                    outputArea.setText("The
                    string \"\" + inputString + "\"\" is rejected
                    by the CFG.");
                }
            }
        });

```

```

        mainPanel.add(simulateButton);

        JButton clearAllButton = new
        JButton("Clear");
        clearAllButton.setFont(new
        Font("Verdana", Font.BOLD, 12));
        clearAllButton.setBounds(325,
        500, 100, 30);
        clearAllButton.setBackground(new
        Color(0x013220));

        clearAllButton.setForeground(Color.white
        );
        clearAllButton.setFocusable(false);

        clearAllButton.setBorder(BorderFactory.c
        reateLineBorder(Color.DARK_GRAY, 1,
        true));
        clearAllButton.addActionListener(
        new ActionListener() {
            @Override
            public void
            actionPerformed(ActionEvent e) {
                inputStringField.setText("");
                outputArea.setText("");
            }
        });
        mainPanel.add(clearAllButton);

        add(mainPanel,
        BorderLayout.CENTER);

        public static void main() {
            SwingUtilities.invokeLater(() -> {
                CFGGUI frame = new
                CFGGUI();
                frame.setVisible(true);
            });
        }
        class CFG {
            public static boolean isWCWR(String
            s) {
                int length = s.length();

                if (length < 3 ||
                Character.toLowerCase(s.charAt(length /
                2)) != 'c') {
                    return false;
                }

                String W = s.substring(0, length /
                2);
                String WR = s.substring((length /
                2) + 1);

                return W.equals(new
                StringBuilder(WR).reverse().toString());
            }

            public static boolean isWE(String s) {
                return s.matches("[ab]*");
            }

            public static boolean
            isAccepted(String s) {
                return isWCWR(s) || isWE(s);
            }
        }
        //tower of hanoi

        import javax.swing.*;
        import java.awt.*;
        import java.util.ArrayList;
        import java.util.List;

        public class TowerOfHanoiVisual extends
        JFrame {
            private int numDisks;
            private final List<Integer>>
            rods;
            private final JPanel canvas;
            private final JTextField diskInputField;
            private final JButton startButton;
            private final JButton resetButton;

            public TowerOfHanoiVisual() {
                this.numDisks = 3;
                this.rods = new ArrayList<>();
                this.canvas = new JPanel() {
                    @Override
                    protected void
                    paintComponent(Graphics g) {

```

```

                        super.paintComponent(g);
                        drawRods(g);
                        drawDisks(g);
                    }
                };

                initializeRods();
                diskInputField = new
                JTextField(10);
                startButton = new
                JButton("Start");
                resetButton = new
                JButton("Reset");

                setupUI();
                setupListeners();
            }

            private void initializeRods() {
                rods.clear();
                for (int i = 0; i < 3; i++) {
                    rods.add(new ArrayList<>());
                }
                for (int i = numDisks; i > 0; i--) {
                    rods.get(0).add(i);
                }
            }

            private void setupUI() {
                this.setTitle("Tower of Hanoi
                Simulator");
                this.setSize(800, 500);

                this.setDefaultCloseOperation(JFrame.DI
                SPOSE_ON_CLOSE);
                this.setLayout(new
                BorderLayout());

                this.getContentPane().setBackground(C
                olor.WHITE);

                JPanel titlePanel = new JPanel();
                titlePanel.setBackground(new
                Color (0xf8f660));

                JLabel titleLabel = new
                JLabel("TOWER OF HANOI
                SIMULATOR");

                titleLabel.setFont(new
                Font("Verdana", Font.BOLD, 24));

                titleLabel.setForeground(Color.black);

                titleLabel.setHorizontalAlignment(Swing
                Constants.CENTER);
                titlePanel.add(titleLabel);

                JPanel controlPanel = new
                JPanel();
                controlPanel.setBackground(new
                Color (0xf8f660));
                JLabel diskLabel = new
                JLabel("Number of Disks:");
                diskLabel.setFont(new
                Font("Verdana", Font.BOLD, 14));

                diskLabel.setForeground(Color.BLACK);
                controlPanel.add(diskLabel);
                controlPanel.add(diskInputField);
                controlPanel.add(startButton);
                controlPanel.add(resetButton);

                diskInputField.setBackground(Color.WHI
                TE);

                diskInputField.setForeground(Color.BLAC
                K);

                diskInputField.setCaretColor(Color.black)
                ;
                startButton.setBackground(new
                Color(0xf8f5b8));

                startButton.setForeground(Color.BLACK)
                ;
                resetButton.setBackground(new
                Color(0xf8f5b8));

```

```

resetButton.setForeground(Color.BLACK);
;

    canvas.setBackground(new
Color(0xf8f5b8));

    this.add(titlePanel,
BorderLayout.NORTH);
    this.add(controlPanel,
BorderLayout.SOUTH);
    this.add(canvas,
BorderLayout.CENTER);

    this.setVisible(true);
}

private void setupListeners() {
startButton.addActionListener(e ->
{
    try {
        int inputDisks =
Integer.parseInt(diskInputField.getText());
    };
    if (inputDisks <= 0) {

JOptionPane.showMessageDialog(this,
"Please enter a number greater than
0.");
    } else {
        numDisks = inputDisks;
        initializeRods();
        canvas.repaint();
        start();
    }
} catch
(NumberFormatException ex) {

JOptionPane.showMessageDialog(this,
"Please enter a valid number.");
    }
});
resetButton.addActionListener(e
-> {
    numDisks = 3;
    initializeRods();
    canvas.repaint();
    });
}

private void drawRods(Graphics g) {
    int rodWidth = 10;
    int rodHeight = 200;
    int baseY = 300;

    g.setColor(Color.black);
    for (int i = 0; i < 3; i++) {
        int x = 150 + i * 200;
        g.fillRect(x, baseY - rodHeight,
rodWidth, rodHeight);
    }
}

private void drawDisks(Graphics g) {
    int baseY =
300;

    int rodXStart = 150;
    int rodSpacing = 200;

    int diskHeight =
Math.min(20, 200 / numDisks);

    for (int i = 0; i < 3; i++)
{
        int rodX = rodXStart +
i * rodSpacing;
        int offsetY = baseY;

        for (int j = 0; j <
rods.get(i).size(); j++) {
            int diskSize =
rods.get(i).get(j);
            int diskWidth =
Math.max(30, diskSize * 20);

            diskWidth =
Math.min(diskWidth, rodSpacing - 20);

            int red = 255;
            int green = 200 +
(diskSize * 15) % 55;
            int blue = 50 +
(diskSize * 10) % 50;
            g.setColor(new
Color(red, green, blue));
            g.fillRect(rodX -
diskWidth / 2, offsetY - diskHeight,
diskWidth, diskHeight);

            g.setColor(Color.WHITE);
            g.drawRect(rodX -
diskWidth / 2, offsetY - diskHeight,
diskWidth, diskHeight);

            offsetY -=
diskHeight;
        }
    }

    private void moveDisk(int from, int
to) {
        int disk =
rods.get(from).remove(rods.get(from).si
ze() - 1);
        rods.get(to).add(disk);
        canvas.repaint();

        try {
            Thread.sleep(600);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void solve(int n, int source,
int auxiliary, int destination) {
        if (n == 1) {
            moveDisk(source, destination);
            return;
        }
        solve(n - 1, source, destination,
auxiliary);
        moveDisk(source, destination);
        solve(n - 1, auxiliary, source,
destination);
    }

    public void start() {
        new Thread() -> {
            solve(numDisks, 0, 1, 2);
        }

JOptionPane.showMessageDialog(this,
"Solved in " + (int) (Math.pow(2,
numDisks) - 1) + " moves!");
    }

    public static void main() {
        SwingUtilities.invokeLater(TowerOfHanoi
Visual::new);
    }
}

//Addition TURING Machine

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class TuringMachineGUI {
    private StringBuilder tape;
    private int head;
    private String state;
    private JTextArea processArea;
    private int decimalResult;

    public TuringMachineGUI(String input,
JTextArea processArea) {
        this.tape = new
StringBuilder(input + " ");
        this.head = 0;
        this.state = "start";

        this.processArea = processArea;
        this.decimalResult = 0;
    }

    public void moveHead(String
direction) {
        if (direction.equals("R")) {
            head++;
        } else if (direction.equals("L")) {
            head--;
        }

        if (head < 0) {
            tape.insert(0, " ");
            head = 0;
        } else if (head >= tape.length()) {
            tape.append(" ");
        }
    }

    public char readSymbol() {
        return tape.charAt(head);
    }

    public void writeSymbol(char symbol)
{
        tape.setCharAt(head, symbol);
    }

    public void execute() {
        displayTapeState();

        while (!state.equals("halt")) {
            char symbol = readSymbol();

            switch (state) {
                case "start":
                    if
(Character.isDigit(symbol)) {
                        state = "addition";
                        moveHead("R");
                    } else {
                        state = "halt";
                    }
                    break;

                case "addition":
                    performBinaryAddition();
                    state = "halt";
                    break;

                default:
                    state = "halt";
            }

            displayTapeState();
        }

        private void performBinaryAddition()
{
            String[] numbers =
tape.toString().trim().split("\\+");
            if (numbers.length == 2) {
                String binary1 =
numbers[0].trim();
                String binary2 =
numbers[1].trim();
                int num1 = Integer.parseInt(binary1,
2);
                int num2 =
Integer.parseInt(binary2, 2);

                decimalResult = num1 + num2;
                String binaryResult =
Integer.toBinaryString(decimalResult);

                tape = new
StringBuilder(binaryResult);
                head = 0;
            }
        }

        private void displayTapeState() {
            StringBuilder display = new
StringBuilder();
            for (int i = 0; i < tape.length();
i++) {
                if (i == head) {
                    display.append("[").append(tape.charAt(
i)).append("]");
                } else {
                    display.append(tape.charAt(i));
                }
            }
        }
    }
}

```

```

    }

processArea.append(display.toString().trim() + "\n");
}

public String getResult1() {
    return tape.toString().trim();
}

public int getDecimalResult() {
    return decimalResult;
}

public static void main() {

    JFrame frame = new
    JFrame("Binary Calculator - Turing
    Machine Simulation");
    frame.setSize(500, 690);

    frame.setDefaultCloseOperation(JFrame.
    DISPOSE_ON_CLOSE);
    frame.setResizable(false);
    frame.setLayout(new
    BorderLayout());

    JPanel headerPanel = new
    JPanel();
    headerPanel.setBackground(new
    Color(0xb5797c));
    headerPanel.setLayout(new
    BoxLayout(headerPanel,
    BoxLayout.Y_AXIS));

    headerPanel.setBorder(BorderFactory.c
    reateEmptyBorder(15, 0, 15, 0));

    JLabel title = new JLabel("BINARY
    ADDITION");
    title.setForeground(Color.white);
    title.setFont(new Font("Verdana",
    Font.BOLD, 28));

    title.setAlignmentX(Component.CENTER
    _ALIGNMENT);

    JLabel subtitle = new
    JLabel("TURING MACHINE
    SIMULATION");

    subtitle.setForeground(Color.white);
    subtitle.setFont(new
    Font("Verdana", Font.PLAIN, 14));

    subtitle.setAlignmentX(Component.CEN
    TER_ALIGNMENT);

    headerPanel.add(title);
    headerPanel.add(subtitle);
    frame.add(headerPanel,
    BorderLayout.NORTH);

    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(null);
    mainPanel.setBackground(new
    Color(0xdec3c5));
    JLabel binary1Label = new
    JLabel("Enter First Binary Number:");

    binary1Label.setFont(new
    Font("Verdana", Font.BOLD, 12));
    binary1Label.setBounds(65, 10,
    200, 30);
    mainPanel.add(binary1Label);
    JTextField binary1Field = new
    JTextField();
    binary1Field.setBounds(260, 10,
    160, 25);
    mainPanel.add(binary1Field);

    JLabel binary2Label = new JLabel("Enter
    Second Binary Number:");
    binary2Label.setFont(new
    Font("Verdana", Font.BOLD, 12));
    binary2Label.setBounds(65, 50,
    200, 30);
    mainPanel.add(binary2Label);

    JTextField binary2Field = new
    JTextField();

```

```

    binary2Field.setBounds(260, 50,
    160, 25);
    mainPanel.add(binary2Field);

    JLabel transitionLabel = new
    JLabel("Results:");
    transitionLabel.setFont(new
    Font("Verdana", Font.BOLD, 13));
    transitionLabel.setBounds(65, 130,
    200, 30);
    mainPanel.add(transitionLabel);

    JPanel roundedPanel = new
    JPanel() {
        @Override
        protected void
        paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d =
            (Graphics2D) g.create();

            g2d.setRenderingHint(RenderingHints.K
            EY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
            g2d.setColor(new
            Color(0xb5797c));
            g2d.fillRoundRect(0, 0,
            getWidth(), getHeight(), 30, 30);

            g2d.setColor(Color.DARK_GRAY);
            g2d.drawRoundRect(0, 0,
            getWidth() - 1, getHeight() - 1, 30, 30);
            g2d.dispose();
        }
    };
    roundedPanel.setLayout(new
    BorderLayout());
    roundedPanel.setBounds(65, 170,
    360, 300);
    mainPanel.add(roundedPanel);

    JTextArea processArea = new
    JTextArea();
    processArea.setFont(new
    Font("Verdana", Font.PLAIN, 12));

    processArea.setBorder(BorderFactory.cr
    eateEmptyBorder(10, 10, 10, 10));
    processArea.setBackground(new
    Color(0xf4e4e5));
    processArea.setEditable(false);
    roundedPanel.add(new
    JScrollPane(processArea),
    BorderLayout.CENTER);

    JButton calculateButton = new
    JButton("Calculate");
    calculateButton.setFont(new
    Font("Verdana", Font.BOLD, 12));
    calculateButton.setBounds(65,
    500, 100, 30);

    calculateButton.setBackground(new
    Color(0xb5797c));

    calculateButton.setForeground(Color.whi
    te);

    calculateButton.setFocusable(false);
    mainPanel.add(calculateButton);

    JButton clearAllButton = new
    JButton("Clear");
    clearAllButton.setFont(new
    Font("Verdana", Font.BOLD, 12));
    clearAllButton.setBounds(325,
    500, 100, 30);
    clearAllButton.setBackground(new
    Color(0xa83840));

    clearAllButton.setForeground(Color.white
    );
    clearAllButton.setFocusable(false);
    mainPanel.add(clearAllButton);

    frame.add(mainPanel,
    BorderLayout.CENTER);

    // Action Listener for Start Button

    calculateButton.addActionListener(new
    ActionListener() {
        @Override
        public void
        actionPerformed(ActionEvent e) {

```

```

            String binary1 =
            binary1Field.getText().trim();
            String binary2 =
            binary2Field.getText().trim();

            if (binary1.isEmpty() ||
            binary2.isEmpty()) {

                JOptionPane.showMessageDialog(frame,
                "Please enter both binary numbers!",
                "Error",
                JOptionPane.ERROR_MESSAGE);
                return;
            }

            String input = binary1 + "+" + binary2;
            processArea.setText("");
            TuringMachineGUI tm = new
            TuringMachineGUI(input, processArea);
            tm.execute();

            processArea.append("\nFinal Binary
            Result: " + tm.getResult1());
            processArea.append("\nFinal
            Decimal Result: " +
            tm.getDecimalResult());
        }
    });

    clearAllButton.addActionListener(new
    ActionListener() {
        @Override
        public void
        actionPerformed(ActionEvent e) {

            the process area

            binary1Field.setText("");
            binary2Field.setText("");
            processArea.setText("");
        }
    });

    frame.setVisible(true);
}
}
//Multiplication Turing Machine

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class UnaryMultiplication {

    public static void main() {

        JFrame frame = new
        JFrame("Unary Multiplication");
        frame.setSize(500, 690);

        frame.setDefaultCloseOperation(JFrame.
        DISPOSE_ON_CLOSE);
        frame.setResizable(false);
        frame.setLayout(new
        BorderLayout());
        JPanel headerPanel = new
        JPanel();
        headerPanel.setBackground(new
        Color(0xb5797c));
        headerPanel.setLayout(new
        BoxLayout(headerPanel,
        BoxLayout.Y_AXIS));

        headerPanel.setBorder(BorderFactory.cr
        eateEmptyBorder(15, 0, 15, 0));

        JLabel title = new JLabel("UNARY
        MULTIPLICATION");
        title.setForeground(Color.white);
        title.setFont(new Font("Verdana",
        Font.BOLD, 28));

        title.setAlignmentX(Component.CENTER
        _ALIGNMENT);

        JLabel subtitle = new
        JLabel("TURING MACHINE
        SIMULATION");

        subtitle.setForeground(Color.white);

```

```

        subtitle.setFont(new
Font("Verdana", Font.PLAIN, 14));

subtitle.setAlignmentX(Component.CEN
TER_ALIGNMENT);

        headerPanel.add(title);
        headerPanel.add(subtitle);
        frame.add(headerPanel,
BorderLayout.NORTH);

```

```

        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(null);
        mainPanel.setBackground(new
Color(0xdec3c5));

```

```

        JLabel unary1Label = new
JLabel("Enter the first unary number:
");
        unary1Label.setFont(new
Font("Verdana", Font.PLAIN, 12));
        unary1Label.setBounds(65, 10,
200, 30);
        mainPanel.add(unary1Label);

```

```

        JTextField unary1Field = new
JTextField();
        unary1Field.setBounds(260, 10,
160, 25);
        mainPanel.add(unary1Field);

```

```

        JLabel unary2Label = new
JLabel("Enter the second unary number:
");
        unary2Label.setFont(new
Font("Verdana", Font.PLAIN, 11));
        unary2Label.setBounds(65, 50,
200, 30);
        mainPanel.add(unary2Label);

```

```

        JTextField unary2Field = new
JTextField();
        unary2Field.setBounds(260, 50,
160, 25);
        mainPanel.add(unary2Field);

```

```

        JLabel resultLabel = new
JLabel("Result:");
        resultLabel.setFont(new
Font("Verdana", Font.BOLD, 13));
        resultLabel.setBounds(65, 130,
200, 30);
        mainPanel.add(resultLabel);

```

```

        JPanel roundedPanel = new
JPanel() {
            @Override
            protected void
paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2d =
(Graphics2D) g.create();

```

```

g2d.setRenderingHint(RenderingHints.K
EY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
g2d.setColor(new
Color(0xb5797c));
g2d.fillRoundRect(0, 0,
getWidth(), getHeight(), 30, 30);

g2d.setColor(Color.DARK_GRAY);
g2d.drawRoundRect(0, 0,
getWidth() - 1, getHeight() - 1, 30, 30);
g2d.dispose();
    }
};
roundedPanel.setLayout(new
BorderLayout());
roundedPanel.setBounds(65, 170,
360, 300);
mainPanel.add(roundedPanel);

```

```

        JTextArea resultArea = new
JTextArea();
        resultArea.setFont(new
Font("Verdana", Font.PLAIN, 12));
        resultArea.setBorder(BorderFactory.crea
teEmptyBorder(10, 10, 10, 10));
        resultArea.setBackground(new
Color(0xf4e4e5));
        resultArea.setEditable(false);
        roundedPanel.add(new
JScrollPane(resultArea),
BorderLayout.CENTER);
        JButton calculateButton = new
JButton("Multiply");
        calculateButton.setFont(new
Font("Verdana", Font.BOLD, 12));
        calculateButton.setBounds(65, 500,
100, 30);
        calculateButton.setBackground(new
Color(0xb5797c));
        calculateButton.setForeground(Color.whi
te);
        calculateButton.setFocusable(false);
        mainPanel.add(calculateButton);
        JButton clearButton = new
JButton("Clear");
        clearButton.setFont(new
Font("Verdana", Font.BOLD, 12));
        clearButton.setBounds(325, 500, 100,
30);
        clearButton.setBackground(new
Color(0xa83840));
        clearButton.setForeground(Color.white);
        clearButton.setFocusable(false);
        mainPanel.add(clearButton);
        frame.add(mainPanel,
BorderLayout.CENTER);
        calculateButton.addActionListener(new
ActionListener() {
            @Override
            public void actionPerformed(ActionEvent
e) {
                String unary1 =
unary1Field.getText().trim();
                String unary2 =
unary2Field.getText().trim();

```

```

if (!isValidUnary(unary1) ||
!isValidUnary(unary2)) {
    resultArea.setText("Invalid input. Unary
numbers must only contain the
character '1.'");
    return; }
String result =
unaryMultiplication(unary1, unary2);
resultArea.setText("Result of
multiplication in unary: " + result +
"\nResult in decimal: " +
result.length());
} });
clearButton.addActionListener(new
ActionListener() {
    @Override
    public void
actionPerformed(ActionEvent e) {
        unary1Field.setText("");
        unary2Field.setText("");
        resultArea.setText(""); } });
frame.setVisible(true); }
private static boolean
isValidUnary(String unary) {
    return unary.matches("1*");}
private static String
unaryMultiplication(String unary1, String
unary2) {
    StringBuilder result = new
StringBuilder();
    for (int i = 0; i < unary1.length(); i++)
    {
        result.append(unary2); }
    return result.toString(); }
}

```